Algorithms (DAA)

Asymptotic Notation

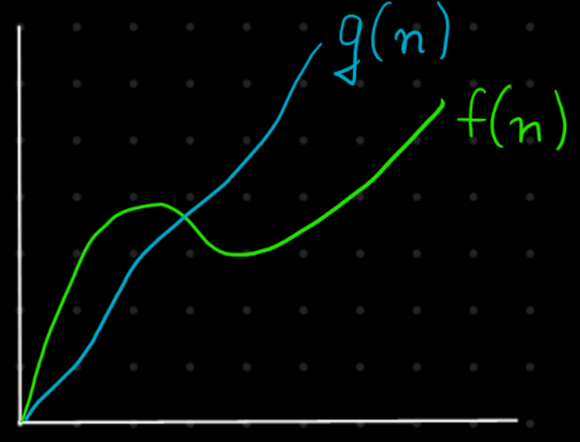
↳ Behavior of a function when the value approaches to ∞.

1) Big O (O)
2) Big Omega (Ω)
3) Theta (Θ)
4) Little O (o)
5) Little Omega (ω)

a) Big O → We have two function $f(n)$ & $g(n)$

$f(n)$ is $O(g(n))$, when
↳ order of

$$f(n) \leq C \cdot g(n) \; \forall \; n \geq K \; \&\& \; \exists C$$



1) Big O (O) / (least upper bound / Tightest upper bound)

$f(n) = O(g(n))$ if and only if

$$f(n) \leq C \cdot g(n) \; \forall \; n \geq K \; \&\& \; \exists C$$

Example - 1) $n^2 + n + 1$, Find $g(n) = ??$

Option → a) n, b) n^2, c) n^3, d) n^4, e) n^5
↳ most accurate

$f(n) \leq g(n)$

$n^2 \leq n^2$

$n^2 + n \leq n^2 + n^2$

$n^2 + n + 1 \leq n^2 + n^2 + n^2$

$f(n) \leq 3 \cdot n^2$

$f(n) \leq C \cdot g(n)$

$$f(n) = O(n^2)$$

$f(n) \leq g(n)$

$n^2 \leq n^3$

$n^2 + n \leq n^3 + n^3$

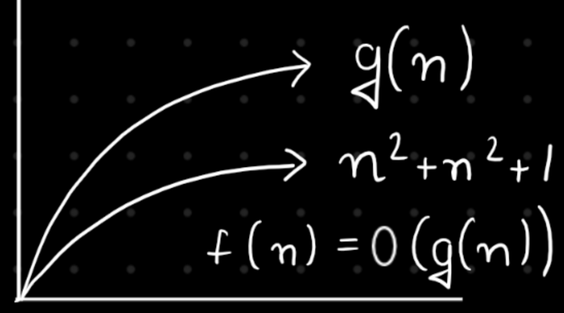
$n^2 + n + 1 \leq n^3 + n^3 + n^3$

$f(n) \leq 3 \cdot n^3$

$f(n) \leq C \cdot g(n)$

$$f(n) = O(n^3)$$

$f(n) = O(n^2) = O(n^3) = O(n^4) = O(n^5)$



2) Big Omega (Ω) (greatest lower bound / tightest lower bound)

$f(n)$ & $g(n)$ are two functions

$f(n) = \Omega(g(n))$ if and only if

$$f(n) \geq C \cdot g(n) \ \forall\, n \geq k \ \&\& \ \exists C$$

Example - $f(n) = n^2 + n + 1$, find $g(n)$

Case - 1)

$f(n) \geq g(n)$

$n^2 \geq n$

$n^2 + n \geq n$

$n^2 + n + 1 \geq n$

$f(n) \geq g(n)$

$$f(n) = \Omega(n)$$

Case - 2

$f(n) \geq g(n)$

$n^2 \geq n^2$

$n^2 + n \geq n^2$

$n^2 + n + 1 \geq n^2$

$f(n) \geq g(n)$

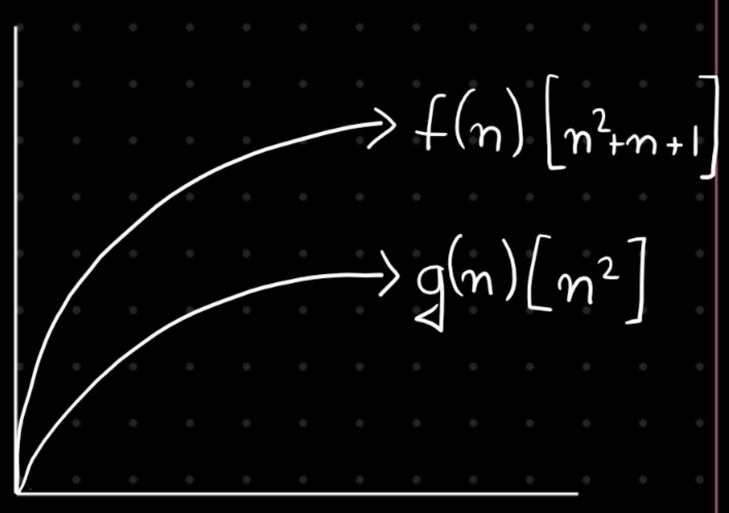
$$f(n) = \Omega(n^2)$$

Case - 3

$f(n) \geq g(n)$

$n^2 \geq n^3 \rightarrow X$

hence $f(n) = \Omega(n) = \Omega(n^2)$



3) Thetha (Θ)

$f(n) = \Theta(g(n))$ if & only if

$$f(n) = O(g(n)) \ \&\& \ f(n) = \Omega(g(n))$$

4) Little O $\rightsquigarrow$

$f(n) = o(g(n)$ if and only if

$$f(n) < C \cdot g(n) \ \forall\, n \geq k \ \&\& \ \forall C$$

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

Example ⇝ $f(n) = n^2$, $g(n) = n^3$

$$\lim_{n \to \infty} \frac{n^2}{n^3} = \frac{1}{n} = \frac{1}{\infty} = 0 \quad \underline{\text{Ans}}$$

5) Little Omega (ω) ⇝

↙ $f(n) = \omega(g(n))$ if and only iff ↘

$$f(n) > C \cdot g(n) \ \forall \ n \geq K \ \& \& \ \forall \ C$$

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$$

Example) $f(n) = n^3$, $g(n) = n^2$

$$\lim_{n \to \infty} \frac{n^3}{n^2} = n = \infty \quad \underline{\text{Ans}}$$

Note

1) $f(n) = O(g(n))$ then $C \cdot f(n) = O(g(n))$

2) $f(n) = O(g(n))$ & $h(n) = O(e(n))$

$$f(n) + h(n) = \max(O(g(n)), O(e(n)))$$

3) $f(n) \cdot h(n) = O(f(n) \cdot h(n))$

Apriori Analysis → Order of magnitude of a statement

↳ # times any statement is executing

Example - 1) $(x = y + z) \rightarrow$ (constant) - C

$O(1) \rightarrow$ Constant time Complexity.

Example - 2) $x = y + z$ — ① ⟶ 1 time

```
for( int i=0 , i < n , i++ ){
    x = y + z  —②  → n time□
```

$i = 0 \quad i = 1 \quad \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \_ \quad i = n-1$ } n times

$x = y + z \quad x = y + z \quad x = y + z$

$(n + 1)$ times $\Rightarrow O(n)$

Example - 3) $x = y + z$ — ① → 1

```
for( int i=0 , i<n , i++ ){
    x = y + z  —②  ⟶ n times.
}
```

```
for ( int i = 0 ; i<n ; i++ ){
    for( int j = 0 ; j < n ; j++ ){
        x = y + z  —  ③  ⟶ n² times
    }
}
```

Example) $n = 5$

```
i = n
while( i > 1):
    i--j
    print("Hello") — (1)
```

$(n-1) \rightarrow O(n)$

Example - 5)

```
i = n
while( i > 1){
    i = i - 3
    print("Hi")
```

$n = 9$	$i = 6$	$i = 3$
$i = 6$	$i = 3$	$i = 0$ ↳ ×
Hi	Hi	

$10/3 = 3.33$ { take upper bound }

Example 6)

$O(\log_2 n)$

```
i = 1
while (i < n):
    i = 2 * i  <-->  i = 3 * i
    print(i)        ↳ O(log_3 n)
```

$O(\log n)$

$n = 64$	$i < 64$	$2 < 64$	$4 < 64$	$8 < 64$
	$i = 2$	$i = 4$	$i = 8$	$i = 16$
	2 ✓	4 ✓	8 ✓	16 ✓
	$16 < 64$	$32 < 64$	$64 \not< 64$	
	$i = 32$	$i = 64$	↳ ×	
	32 ✓	64		

Example - 7)
```
i = n
while i > 2:
    i = i^1/2
```

$n = 256 \rightarrow 256 > 2 \rightarrow 16 > 2 \rightarrow 4 > 2 \rightarrow 2 \not> 2$ (└ x)

$i = 256 \quad i = 16 \quad i = 4 \quad i = 2$

$n^{1/2^k} = 2$ ↪ stopping criteria

$\Rightarrow \log_2 n^{1/2^k} = \log_2 1 \Rightarrow \frac{1}{2^k} \log_2 n = 1 \Rightarrow \log_2 n = 2^k$

$\log(\log_2 n) = \log_2 2^k \Rightarrow \log(\log_2 n) = k \log_2 1$

$K = \log(\log_2 n) \hookrightarrow O(\log(\log_2 n))$

Complexcity Class

1) Constant - $O(1)$
2) Logarithmic - $O(\log n)$ ⟶ $O(\log(\log n))$
3) Linear - $O(n)$
4) Quadratic - $O(n^2)$
5) Cubic - $O(n^3)$
6) Polynomial - $O(n^c)$
7) Exponential - $O(c^n)$

Increasing Order time Complexcity ↓

8) $n! < n^n$

$2^n < n^n$

$n! > 2^n$

$$2^n < n! < n^n$$

9) $\log n < n$

$(\log n)^2 < n$

$(\log n)^3 < n$

⋮

$(\log n)^{1000} < n$

→ But $(\log n)^{\log n} > n$ → True

Recurrence Relation

Practice Sheet - IV

Q) Solve questions with Substitution or recursive Tree Approch.

1) $T(n) = 2T(n-1)$, $T(1) = 1$

$\Rightarrow T(n-1) = 2T(n-1-1) = 2T(n-2)$

$\Rightarrow T(n) = 4T(n-2)$

$\Rightarrow T(n-2) = 4T(n-3)$

$\Rightarrow T(n) = 8T(n-3)$

⋮ K times

$T(n) = 2^k T(n-k)$, where $n-k=1$

$\Rightarrow k = n-1$

$\Rightarrow T(n) = 2^{(n-1)} T(n-n-1)$

$T(n) = 2^{(n-1)} T(1)$; $T(1) = 1$

$T(n) = 2^{(n-1)}$ $\Rightarrow O(2^n)$ Ans

11) $T(n) = T(n/2) + n$; $T(1) = 1$

$$T(n) = T(n/2) + n$$

$$\Rightarrow T(n/2) = T(n/4) + \frac{n}{2}$$

$$\Rightarrow T(n) = T(n/4) + n/2 + n$$

$$\Rightarrow T(n/4) = T(n/8) + n/4$$

$$\Rightarrow T(n) = T(n/8) + n/4 + n/2 + n$$

$\vdots$ k times

$$T(n) = T(n/2^k) + n/2^{k-1} + n/2^{k-2} + \ldots + \frac{n}{2^2} + \frac{n}{2} + n$$

Now $\frac{n}{2^k} = 1 \Rightarrow n = 2^k$

$\Rightarrow \log_2 n = k \log_2 2 \Rightarrow k = \log_2 n$

$$\Rightarrow T(n) = T\left(\frac{n}{2^{\log_2 n}}\right) + \frac{n}{2^{\log n - 1}} + \frac{n}{2^{\log n - 2}} + \ldots + n$$

$$\Rightarrow T(1) + \frac{n}{2^{\log_2 n}/2} + \frac{n}{2^{\log_2 n}/2^2} + \frac{n}{2^{\log_2 n}/2^3} + \ldots + n$$

$$\Rightarrow T(1) + 2 + 2^2 + 2^3 + \ldots + n$$

$$\Rightarrow 2^0 + 2^1 + 2^2 + 2^3 + \ldots + n$$

applying sum of GP $\rightsquigarrow a \cdot \frac{r^K - 1}{r - 1}$

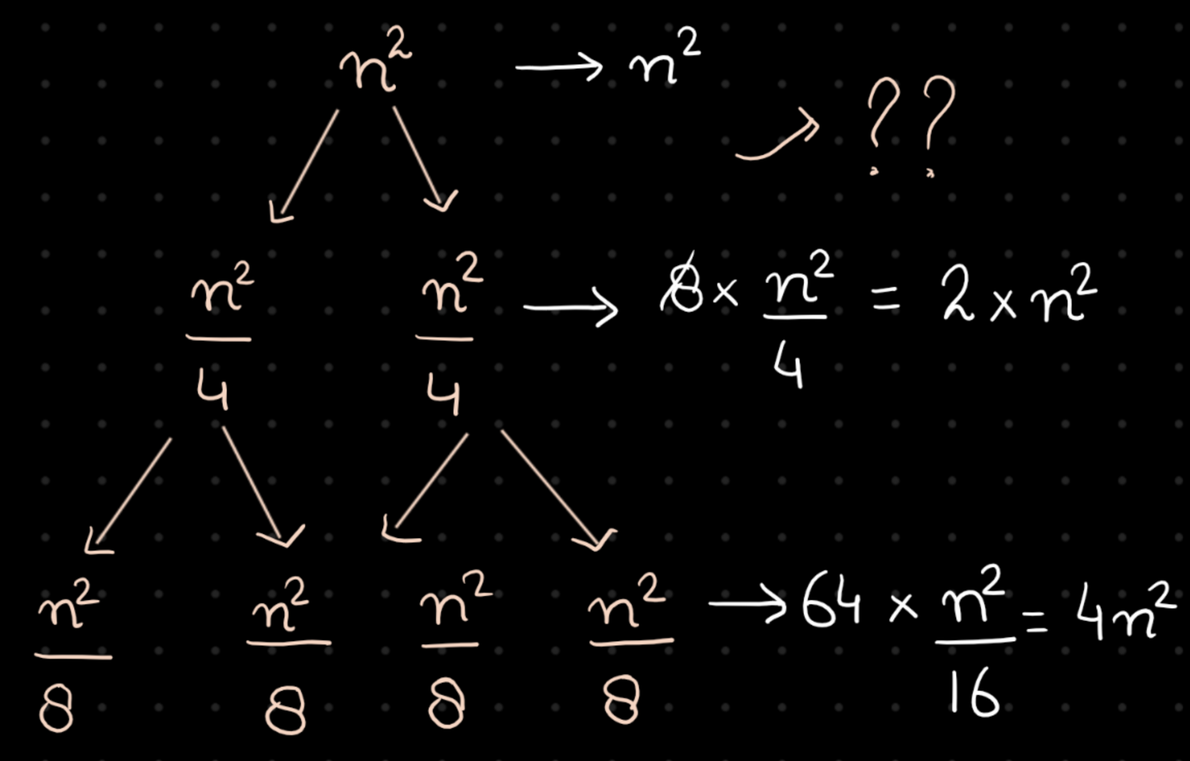
$$= 1 \cdot \frac{\left(2^{\log_2 n} - 1\right)}{2 - 1} = (n-1) \Rightarrow O(n)$$ ✗

iii) $T(n) = 8T(n/2) + n^2$, $T(1) = 1$ → ill redo this

$$\Rightarrow 4T(n/2) + 4T(n/2) + n^2$$

Note
cost per level
can/ cannot be
same in each
level

in our case
- node $\rightsquigarrow 8^i$
- Each node cost
  - $\hookrightarrow (n/2^i)^2 = n^2/4^i$
- Level cost
  - $\hookrightarrow 8^i \times \frac{n^2}{4^i} = n^2 \cdot 2^i$



left side $\rightarrow \frac{n^2}{2^K} = 1$

$\rightarrow$ same

Right side $\rightarrow \frac{n^2}{2^K} = 1$

$$n^2 = 2^K \Rightarrow \log_2 n^2 = K \log_2 1$$

$$K = \log_2 n^2$$

iii) $T(n) = 8\,T(n/2) + n^2 \; ; \; T(1) = 1$

$\Rightarrow T(n/2) = 8T\left(\frac{n}{4}\right) + \frac{n^2}{4}$

$\Rightarrow T(n) = 64\,T\left(\frac{n}{4}\right) + \frac{n^2}{4} + n^2$

$\Rightarrow T(n/4) = 64\,T\left(\frac{n}{16}\right) + \frac{n^2}{16}$

$\Rightarrow T(n) = 8^4\,T\left(\frac{n}{16}\right) + \frac{n^2}{16} + \frac{n^2}{4} + n^2$

⋮

k time

$T(n) = 8^k\,T\left(\frac{n}{2^k}\right) + \frac{n^2}{2^k} + \frac{n^2}{2^{k-1}} + \ldots\ldots + n^2$

$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow \log_2^n = k$

$T(n) = 8^{\log_2^n}\,T\left(\frac{n}{2^{\log_2^n}}\right) + \frac{n^2}{2^{\log_2^n - 1}} + \ldots + n^2$

$\Rightarrow 8^{\log_2^n} \cdot T(1) + 2n + 4n + \ldots + n^2$

$\Rightarrow n^3 + 2n + 4n + 8n + \ldots + n^2$

$\Rightarrow n^3 + \ldots$ some gp $\rightsquigarrow O(n^3)$ Ans

(iv) $T(n) = 2T(n/2)\ ;\ T(1) = 1$

$$T(n) = 2T(n/2)$$

$$T(n/2) = 2T(n/4)$$

$$\Rightarrow T(n) = 4T(n/4)$$

$$\Rightarrow T(n/4) = 4T(n/16)$$

$$\Rightarrow T(n) = 16T(n/16)$$

$$\vdots$$

k time

$$T(n) = 2^k\left(n/2^k\right) \rightsquigarrow \frac{n}{2^k} = 1 \Rightarrow n = 2^k$$

$$\Rightarrow k = \log n$$

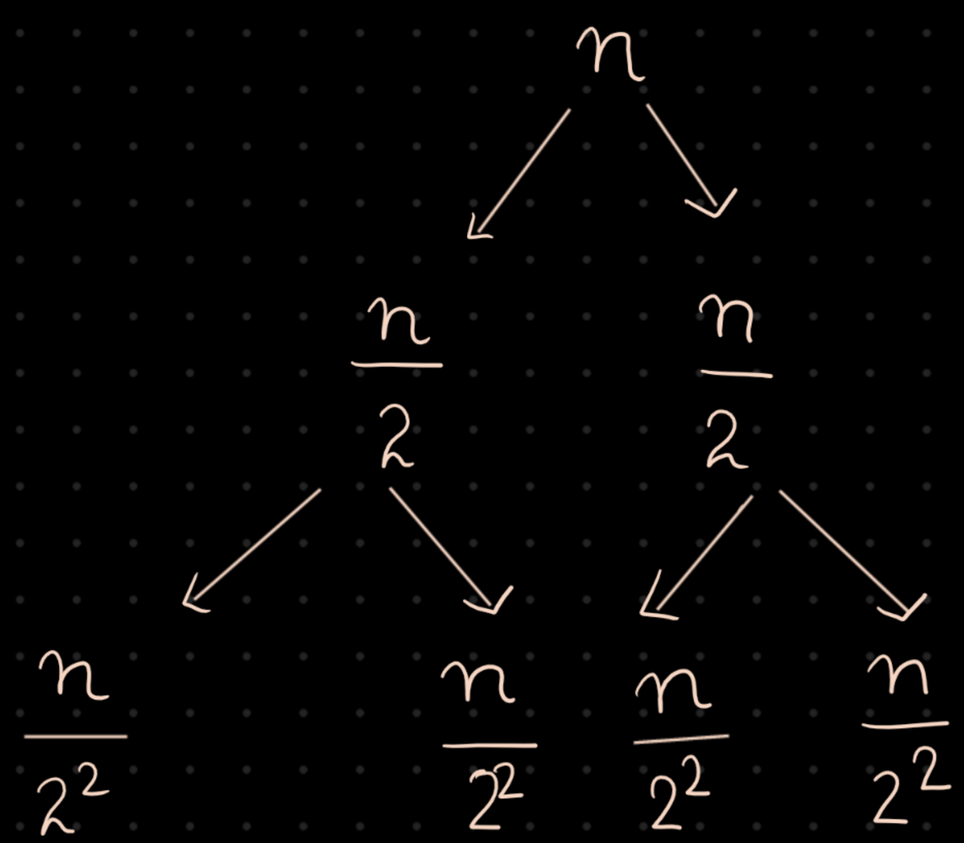
$$\Rightarrow 2^{\log_2 n}\left(n/2^{\log_2 n}\right)$$

$$\Rightarrow n \cdot 1 = O(n)\ \underline{\text{A}}$$

v) $T(n) = 2T(n/2) + 1\ ,\ T(1) = 1$

↳ Same as above $O(n)$

vi) $T(n) = 2T(n/2) + n$



K time

Left side

$$\frac{n}{2^k} = 1$$

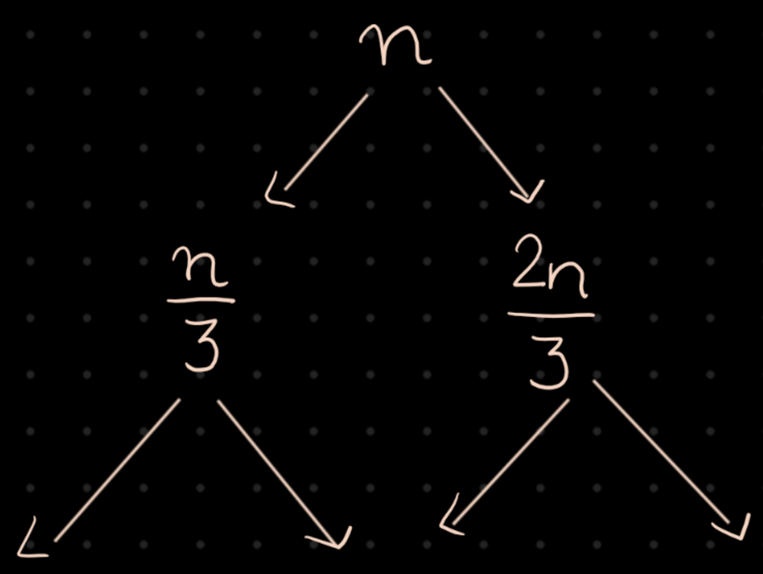
Right side

$$\frac{n}{2^k} = 1$$

$K = \log n$ [In both side]

$\rightarrow O(n * K) \rightsquigarrow O(n \log n)$ Ans.

vii) $T(n) = T(n/3) + T(2n/3) + n$, $T(1) = 1$



$\frac{n}{3^2}$ $\quad$ $\frac{2n}{3^2}$ $\quad$ $\frac{2n}{3^2}$ $\quad$ $\frac{2^2 n}{3^2}$

⋮ K time

Left side

$$\frac{n}{3^{K_L}} = 1 \Rightarrow K = \log_3 n$$

Right side

$$\frac{n}{(3/2)^{K_R}} = 1 \Rightarrow K = \log_{3/2} n$$

since $\log_{3/2} n > \log_3 n$

$$\Rightarrow O(n * K) = O(n \log_{3/2} n) \quad \underline{\text{Ans}}$$

VIII) $T(n) = 5 \cdot T(n/5) + C \cdot n$,

$$T(n) = 5 \cdot T(n/5) + C \cdot n$$

$$T(n/5) = 5 \cdot T(n/5^2) + C \cdot \frac{n}{5}$$

$$T(n) = 5\left(5 \cdot T(n/5^2) + C \cdot \frac{n}{5}\right) + Cn$$

$$\Rightarrow 5^2 T\left(\frac{n}{5^2}\right) + 2Cn$$

$$T(n/5^2) = 5 \cdot T\left(\frac{n}{5^3}\right) + \frac{n}{5^2}$$

$$T(n) = 5^3 \cdot T\left(\frac{n}{5^3}\right) + 3 \cdot Cn$$

$$T(n) = 5^K \cdot T\left(\frac{n}{5^K}\right) + K \cdot cn$$

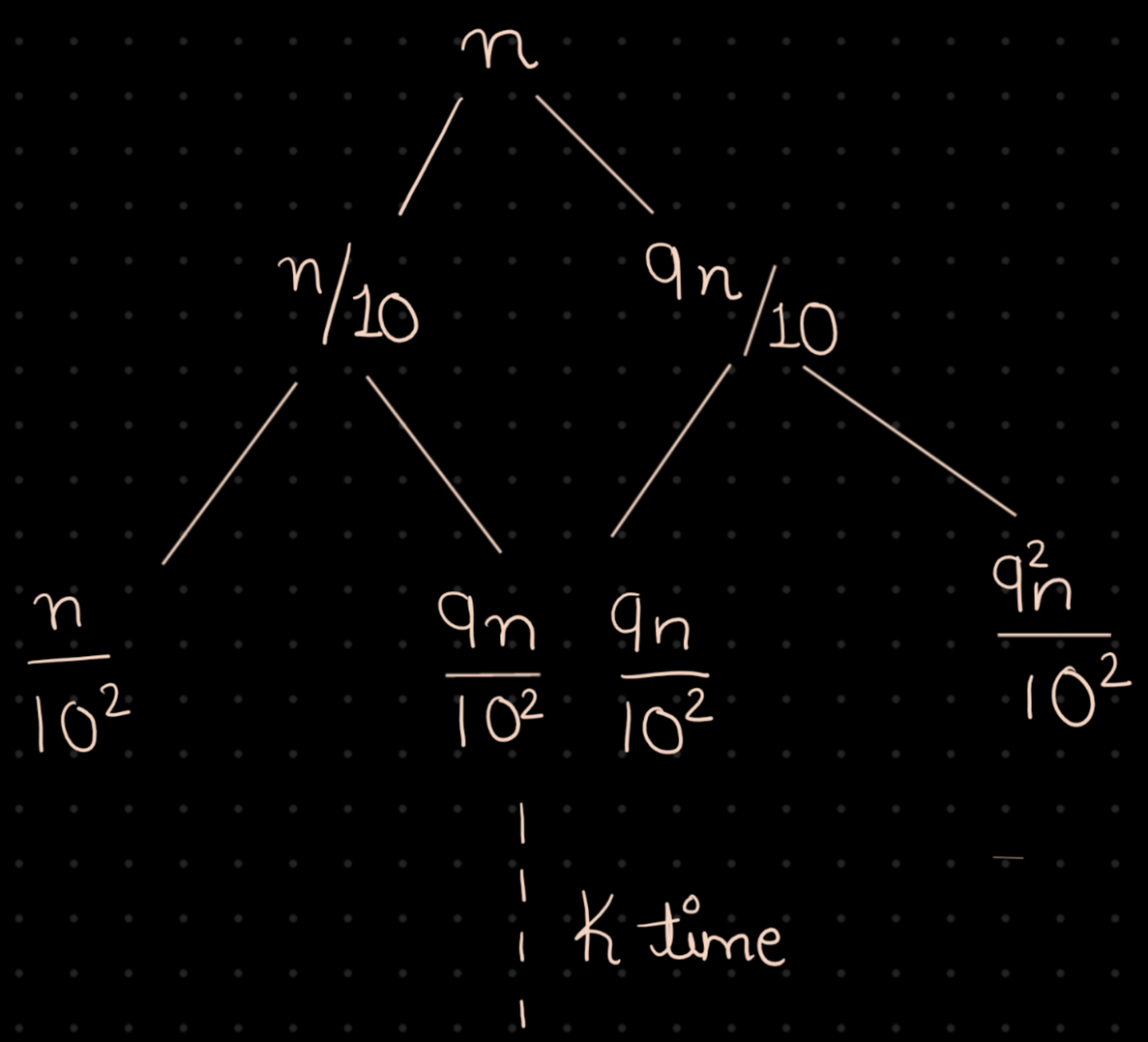
$$\frac{n}{5^K} = 1 \Rightarrow n = 5^K \Rightarrow K = \log_5 n$$

$$T(n) = 5^{\log_5 n} \cdot T\left(\frac{n}{5^{\log_5 n}}\right) + \log_5 n \cdot n$$

$$= n \cdot T(1) + n \log_5 n$$

$$= n + n \log_5 n \Rightarrow O(n \log n) \text{ Ans}$$

9) $T(n) = T(n/10) + T(9n/10) + n$



Left Side

$$\frac{n}{10^k} = 1$$

$$k = \log_{10}^{n}$$

Right Side

$$\frac{n}{\left(\frac{10}{9}\right)^k} = 1,$$

$$\Rightarrow k = \log_{10/9}^{n}$$

$$\Rightarrow \log_{10/9}^{n} > \log_{10}^{n}$$

$$\Rightarrow O(n * k) \Rightarrow O(n * \log_{10/9}) = O(n \log n)$$ ∴